

29 — Semantic Re-ranking

Requirement: R13, R14 (hybrid search with RRF fusion) **Phase:** M

What it is

When you search for something in this system, the results go through a two-stage pipeline. The first stage uses **Reciprocal Rank Fusion (RRF)** to combine vector search and BM25 keyword search into a single ranked list. The second, optional stage is **semantic re-ranking** — it takes those RRF results and adjusts their order using actual cosine similarity scores between the query and each result's embedding.

Here is why the second stage exists and how it works:

The RRF limitation

RRF is a brilliant algorithm for combining ranked lists from different sources, but it has a deliberate trade-off: **it throws away the actual similarity scores and only uses rank positions.** If vector search returns items A, B, C ranked 1st, 2nd, 3rd, RRF only knows that A beat B and B beat C. It does not know by how much.

This is actually a feature, not a bug, for the initial fusion step. Vector cosine similarity scores (0.0 to 1.0) and BM25 `ts_rank` scores (arbitrary positive numbers) live on completely different scales. You cannot meaningfully average 0.85 cosine similarity with 3.72 BM25 rank — the numbers are not comparable. RRF avoids this problem entirely by converting everything to ordinal ranks.

But once the lists are fused, you sometimes want to refine the order using semantic meaning. Two items might be tied at the same RRF rank, but one might be 0.95 cosine similar to the query while the other is only 0.72. Re-ranking recovers this lost information.

Cosine similarity

Cosine similarity measures the angle between two vectors in high-dimensional space. It answers the question: "How similar is the direction these two vectors point in?"

The formula is: $\cos(\theta) = (A \cdot B) / (|A| * |B|)$

- If two vectors point in exactly the same direction: cosine similarity = 1.0
- If they are perpendicular (completely unrelated): cosine similarity = 0.0
- If they point in opposite directions: cosine similarity = -1.0

For text embeddings, cosine similarity captures semantic meaning: "industrial pump manufacturer" and "ipari szivattyugyarto" (the Hungarian equivalent) will have high cosine similarity even though they share zero words, because the embedding model learned that these phrases mean the same thing.

The blending formula

The re-ranking step combines RRF score and cosine similarity with a weighted blend:

```
reranked_score = (1 - weight) * rrf_score + weight * cosine_similarity
```

With the default weight of 0.3:

```
reranked_score = 0.7 * rrf_score + 0.3 * cosine_similarity
```

This keeps RRF as the dominant signal (70%) while letting cosine similarity break ties and make fine-grained adjustments (30%). RRF already proved these results belong in the top-N; cosine similarity refines which of those top results should rank higher.

Parsing stored embeddings

The cosine similarity calculation requires the original embedding vectors for each result. These are stored in PostgreSQL as the `embedding` column. The `_parse_pg_vector` function handles the fact that PostgreSQL might return the vector in different formats (as a Python list, a string like `"[0.1,0.2,0.3]"`, etc.) depending on the database driver and pgvector version.

Real-life analogy

Imagine you are judging a cooking competition with two panels of judges.

Panel A (vector search) scores each dish on flavor complexity, on a 0-100 scale. **Panel B (BM25)** scores based on how well the dish matches the stated theme, using a completely different scoring system (letter grades A through F).

RRF is like converting both panels' results to simple rankings: "Panel A ranked Chef Alice 1st, Chef Bob 2nd, Chef Carol 3rd. Panel B ranked Chef Carol 1st, Chef Alice 2nd, Chef Bob 3rd." You combine the rankings without worrying about the different scoring scales. After RRF, Alice and Carol might be nearly tied.

Semantic re-ranking is like bringing in a head judge who tastes each finalist's dish and says, "Alice's dish is 95% aligned with what I was looking for; Carol's is 88%." The head judge's tasting is the cosine similarity — a direct, meaningful comparison between the query (what you wanted) and each result (what you got). This breaks the tie: Alice moves to 1st.

The 70/30 blend means you trust the two panels' combined wisdom for most of the ranking, but the head judge gets a meaningful voice to refine the final order.

Why we used it here

RRF is required by the project spec (R13, R14) and is the primary fusion algorithm — it is never a weighted average of raw scores (CLAUDE.md explicitly forbids that). But RRF's rank-only nature means it cannot distinguish between a result that is 0.98 similar to your query and one that is 0.72 similar, if they happen to land at the same rank position.

In a bilingual search system, this matters: a query in Hungarian might find excellent matches in both languages, and the subtle ranking differences between them are captured in the embedding space but lost by RRF. Re-ranking recovers that signal.

The re-ranking is **opt-in** (controlled by a `rerank` parameter) because it adds computational cost — it requires loading the full embedding vectors for every result, converting them to numpy arrays, and computing dot products. For most searches, RRF alone is good enough. For cases where result quality matters most (like the Admin UI's search test page), users can toggle re-ranking on.

Code walkthrough

1. Parsing PostgreSQL vector strings (`search_api/services/search.py`)

Before computing cosine similarity, the stored embeddings need to be converted from their database representation to Python lists:

```
def _parse_pg_vector(emb) -> list[float] | None:
    # pgvector can return embeddings in different formats depending
    # on the driver version and cursor factory:
    if isinstance(emb, (list, tuple)):
        # Already a Python sequence – just convert to list
        return list(emb)
    if isinstance(emb, str):
        # String format like "[0.123,0.456,0.789]"
        # Strip brackets and whitespace, then split on commas
        s = emb.strip("[ ] ")
        if s:
            return [float(x) for x in s.split(",")]
    # Null or unrecognized format – cannot compute similarity
    return None
```

2. The re-ranking function (`search_api/services/search.py`)

This is the core of semantic re-ranking — it computes cosine similarity between the query embedding and each result's stored embedding, then blends it with the existing RRF score:

```
def _rerank_by_cosine(
    results: list[dict],
    query_vec: list[float],
    weight: float = 0.3,
) -> list[dict]:
    if not results or not query_vec:
        return results # Nothing to rerank

    import numpy as np

    # Convert the query vector to a numpy array and normalize it.
    # Normalizing means dividing by the vector's length (L2 norm)
    # so it has magnitude 1.0. This simplifies cosine similarity
    # to just a dot product: cos(A, B) = A_norm dot B_norm
    q = np.array(query_vec, dtype=np.float32)
```

```

q_norm = np.linalg.norm(q)
if q_norm == 0:
    return results # Zero vector – cannot compute similarity
q = q / q_norm # Now |q| = 1.0

for r in results:
    # Extract the stored embedding from the result row.
    # During search_table(), the embedding column was renamed
    # to "_embedding" to avoid sending raw vectors to the client.
    emb = _parse_pg_vector(r.get("_embedding"))
    if emb is not None:
        d = np.array(emb, dtype=np.float32)
        d_norm = np.linalg.norm(d)
        # Cosine similarity = dot(q_normalized, d) / |d|
        # Since q is already normalized, this simplifies to:
        # dot(q, d) / |d| which equals dot(q, d/|d|)
        cos_sim = float(np.dot(q, d) / d_norm) if d_norm > 0 else 0.0
    else:
        cos_sim = 0.0 # No embedding available – use zero

    # Get the existing RRF score from the fusion step
    rrf = r.get("rrf_score", 0.0)

    # Store both scores on the result for transparency
    r["cosine_similarity"] = round(cos_sim, 5)

    # Blend: 70% RRF + 30% cosine similarity
    # RRF is the dominant signal (it already proved these results
    # are relevant across both vector and keyword search).
    # Cosine similarity refines the order within the top results.
    r["reranked_score"] = round(
        (1 - weight) * rrf + weight * cos_sim, 5
    )

    # Re-sort by the blended score, highest first
    results.sort(key=lambda r: r["reranked_score"], reverse=True)
return results

```

3. Integration in the search pipeline (`search_api/services/search.py`)

Re-ranking is triggered conditionally at the end of the main `search_table` function:

```

def search_table(table, query, limit=20, offset=0, ..., rerank=False):
    # ... (vector search, BM25 search, RRF fusion all happen first) ...

```

```

# When fetching results, keep the embedding if reranking is requested
for item_id, score in page:
    row = rows_by_id.get(item_id)
    if row:
        row["rrf_score"] = score
        if rerank and "embedding" in row:
            # Rename to "_embedding" – keep it for re-ranking,
            # but signal that it should not be sent to the client
            row["_embedding"] = row.pop("embedding")
        elif "embedding" in row:
            # Not re-ranking – drop the embedding entirely
            # (it is large and the client does not need it)
            del row["embedding"]
        results.append(row)

retrieval_mode = "hybrid_rrf"
if rerank and results:
    # Run the re-ranking pass with the query embedding
    results = _rerank_by_cosine(results, query_vec)
    retrieval_mode = "hybrid_rrf_reranked"

# Clean up: remove the raw embeddings before returning to client
for r in results:
    r.pop("_embedding", None)

return {
    "retrieval_mode": retrieval_mode, # tells the client what happened
    "results": results,
    # ...
}

```

The `retrieval_mode` field in the response switches from `"hybrid_rrf"` to `"hybrid_rrf_reranked"` so the client knows whether re-ranking was applied. The Admin UI's search test page displays this as a badge alongside each result's RRF score, cosine similarity, and reranked score.

How to explain this in an interview

"Our search pipeline uses a two-stage ranking approach. The first stage is Reciprocal Rank Fusion, which safely combines vector search and BM25 keyword search by converting their incomparable scores into ordinal ranks — this avoids the pitfall of naively averaging cosine similarity with BM25 scores that live on different scales. The second stage is an optional semantic re-ranking pass: we

load the stored embedding vectors for each RRF result, compute cosine similarity against the query embedding using numpy, and blend the RRF score (70%) with cosine similarity (30%) to produce a refined ranking. This recovers the fine-grained semantic similarity information that RRF intentionally discards, letting us break ties and adjust the order of closely-ranked results while still respecting the broad ranking that RRF established."